

```
// OSSP Skill 19 - SIGINT Handling
// Demonstrates signal handler registration, protecting the
// shell from Ctrl+C, forwarding SIGINT to a foreground child,
// and updating foreground state.

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

static volatile sig_atomic_t foreground_pid = -1;

static void handle_sigint(int sig) {
    (void)sig;

    if (foreground_pid > 0) {
        kill(foreground_pid, SIGINT);
        write(STDOUT_FILENO,
            "\nSIGINT forwarded to foreground process.\n", 42);
    } else {
        write(STDOUT_FILENO,
            "\nShell protected from SIGINT. No foreground job.\n", 50);
    }
}

int main(void) {
    struct sigaction sa;
    sa.sa_handler = handle_sigint;
```

```
int main(void) {
    struct sigaction sa;
    sa.sa_handler = handle_sigint;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = 0;

    if (sigaction(SIGINT, &sa, NULL) == -1) {
        perror("sigaction");
        return 1;
    }

    printf("== Skill 19: SIGINT Handler ==\n");
    printf("The parent handles Ctrl+C and forwards it to the child.\n");

    pid_t pid = fork();

    if (pid == -1) {
        perror("fork");
        return 1;
    }

    if (pid == 0) {
        signal(SIGINT, SIG_DFL);
        printf("Foreground child pid=%d. Press Ctrl+C.\n", getpid());
        fflush(stdout);

        for (;;) {
            printf("Child running...\n");

            for (;;) {
                printf("Child running...\n");
                fflush(stdout);
                sleep(1);
            }
        }

        foreground_pid = pid;

        int status;
        waitpid(pid, &status, 0);

        foreground_pid = -1;

        printf("Foreground child terminated. Shell continues safely.\n");
        return 0;
    }
}
```

```
sai_pavani@Sai-Pavani:~/OS_SKILL$ nano skill_19.c.c
sai_pavani@Sai-Pavani:~/OS_SKILL$ gcc skill_19.c.c -o skill_19
sai_pavani@Sai-Pavani:~/OS_SKILL$ ./skill_19
=== Skill 19: SIGINT Handler ===
The parent handles Ctrl+C and forwards it to the child.
Foreground child pid=1552. Press Ctrl+C.
Child running...
Child running...
Child running...
Child running...
Child running...
^C
SIGINT forwarded to foreground process.
Foreground child terminated. Shell continues safely.
sai_pavani@Sai-Pavani:~/OS_SKILL$ |
```